LAB EXERCISE: 5

Word Sense Disambiguation and Induction with WordNet and Lexical Relations — SenseLens WSD & WSI Studio

A Python / Flask web application, command-line tool and Jupyter notebook that lists the senses of an ambiguous word from WordNet, explores their lexical relations, and decides which sense a sentence uses with five disambiguation methods and an unsupervised sense-induction module.

Aim / Objective

This project develops a **word sense disambiguation (WSD)** and **word sense induction (WSI)** system using **WordNet and lexical relations**, as set in the problem statement of the lab. WordNet groups words into *synsets* — sets of synonyms that share one meaning — so each synset of a polysemous word such as “bank” is one of its senses, with a definition (gloss), example sentences and relations (hypernyms, hyponyms, meronyms, holonyms, antonyms) to other concepts.

The project is called **SenseLens**. The code is split into a `wsd_engine/` package (`wordnet_explorer.py`, `wsd_algorithms.py`, `supervised_wsd.py`, `sense_induction.py`, `benchmark_data.py`, `evaluator.py`), a Flask dashboard and REST API (`app.py`), a command-line tool (`run_wsd_cli.py`), a unit-test suite (`test_wsd.py`) and a lab notebook (`ANLP_WSD_Lab_Report.ipynb`). The evaluator measures how well each method picks the gold sense on 20 hand-annotated benchmark sentences, so that the answers the system gives can be trusted to a known degree.

Expected Outcomes Addressed

- **1. Identify different senses of ambiguous words.** For any word the WordNet sense inventory is listed with synset ID, part of speech, definition, examples, lemma frequencies and relation counts; “bank” has 18 senses (10 noun, 8 verb). The inventory can be exported to JSON and CSV.
- **2. Use WordNet to explore lexical relations.** Hypernym paths up to the root, hyponyms, part / member / substance meronyms and holonyms, antonyms, the lowest common subsumer (LCS) and Path / Wu-Palmer similarity are available for every synset.
- **3. Apply WSD algorithms to sample sentences.** Classic (simplified) Lesk, Extended (adapted) Lesk, Cosine TF-IDF Lesk, Wu-Palmer similarity and a supervised Linear SVM disambiguate a target word, with the overlapping words and a confidence shown; an “all content words” mode tags every polysemous word of a sentence.
- **4. Induce senses without a dictionary.** K-Means (or agglomerative) clustering of context vectors discovers sense clusters from unannotated sentences and aligns each cluster to a WordNet synset.
- **5. Compare methods and report limitations honestly.** The five WSD methods are scored on 20 gold sentences, with a sentence-by-sentence matrix, a per-word breakdown and a discussion of what the small benchmark can and cannot show.

Tools & Technologies Used

Category	Tool / Library	Purpose in this project
Language / runtime	Python 3.10+	Core implementation of sense lookup, WSD, WSI and evaluation.
Lexical database	NLTK WordNet 3.0 (wordnet, omw-1.4)	Synsets, glosses, examples, lemma counts, hypernym / hyponym / meronym / holonym / antonym relations, LCS, Path and Wu-Palmer similarity.

NLP preprocessing	NLTK tokeniser, stop words, WordNetLemmatizer; spaCy (en_core_web_sm)	Tokenise and lemmatise sentences, remove stop words, tag parts of speech of the context.
Machine learning	scikit-learn (TfidfVectorizer, LinearSVC, MultinomialNB, KMeans, PCA, cross_val_score)	TF-IDF vectors, supervised sense classifier, K-Means context clustering, cross-validation.
Data handling / plots	NumPy, pandas, Matplotlib, seaborn	Sense-inventory tables, accuracy charts in the notebook and this report.
Web framework	Flask 3 (REST API + HTML / CSS / JavaScript front end)	Five-tab dashboard: WordNet Explorer, WSD Disambiguator, Sense Induction, Benchmarks, Analysis Report.
Testing	unittest (test_wsd.py)	Automated checks of the explorer, algorithms, supervised model, WSI and evaluator.

Dataset / Corpus

Lexicon. WordNet 3.0 (Princeton) is the knowledge source (117,659 synsets of nouns, verbs, adjectives and adverbs). A word with several synsets is ambiguous: “bank” alone has 18.

Benchmark sentences. Twenty sentences covering seven polysemous nouns (bank, plant, bass, crane, apple, mouse, star) are stored in `data/wsd_benchmark_sentences.json`. Each sentence is annotated with its gold WordNet synset, and most targets have one sentence for each of two contrasting senses (for example plant.n.01 “industrial plant” vs plant.n.02 “flora”). The set is small and was labelled by the project authors, so it is a demonstration benchmark rather than a standard test set.

Supervised training corpus. `data/supervised_wsd_corpus.json` holds labelled example sentences for four words only (bank, plant, bass, crane), used to train one Linear SVM per word.

Property	Value
Benchmark set	20 sentences; targets bank 4, plant 4, bass 4, crane 2, apple 2, mouse 2, star 2
Supervised model coverage	bank, plant, bass, crane (14 of the 20 sentences); apple, mouse and star have no trained model (shown as N/A)
WSI demo corpus	8 context sentences for “bank”, K = 2 clusters
Sense inventory of “bank”	18 synsets: 10 noun + 8 verb

System Architecture / Pipeline Design

Stage	What happens
1. Input	A word, an optional part-of-speech filter and, for the disambiguator, one context sentence.
2. WordNet lookup	get_sense_inventory collects every synset of the word with its POS, gloss, examples, lemmas and SemCor-style and lemma counts.
3. Relations	get_lexical_relations resolves hypernym paths to entity.n.01, hyponyms, meronyms, holonyms, antonyms and derivational forms; compare_synset_similarity gives LCS, Path and Wu-Palmer scores.
4. Explorer view	Sense cards with relation counts, a Lexical Relations drawer, and JSON / CSV export.

Stage	What happens

5. Preprocess	The sentence is tokenised, lower-cased, stop words removed and tokens lemmatised; the target word is located.
6. Sense signatures	For each candidate synset: gloss + examples (classic); plus hypernym, hyponym and meronym glosses with the own gloss counted twice (extended); TF-IDF vectors (cosine).
7. Scoring	Overlap count (Lesk), cosine similarity, mean Wu-Palmer similarity to context nouns, or a Linear SVM over collocation window w(-2..+2) and n-gram TF-IDF features. The highest-scoring synset wins. Context vectors are clustered with K-Means for WSI.
8. Result view	Chosen synset and definition, overlapping words highlighted, a confidence bar per method, a benchmark matrix and an analysis report (Markdown / PDF).

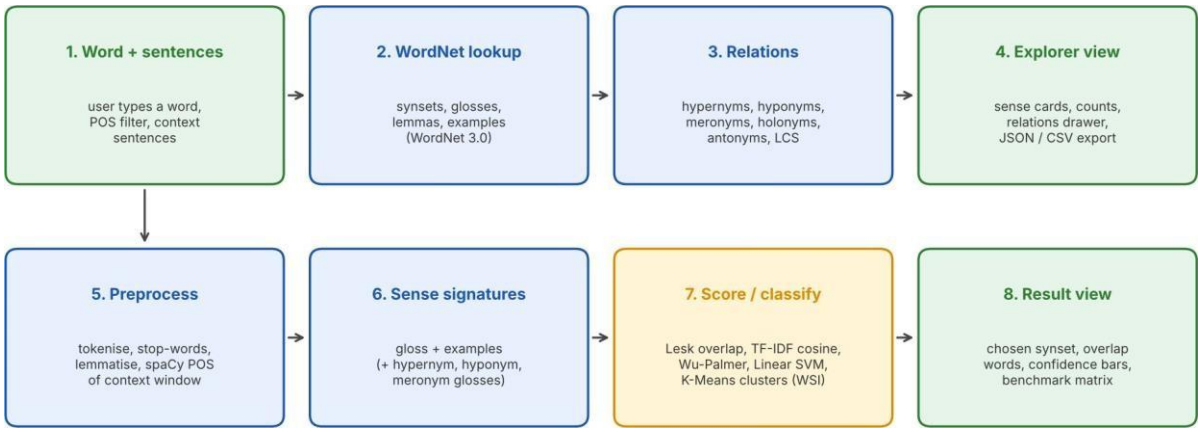


Figure 1. *Top row: dictionary lookup and relation exploration (stages 1–4). Bottom row: context-based disambiguation and induction (stages 5–8). Blue = NLP processing, amber = scoring, green = what the user sees.*

Backend Implementation

6.1 Approach / Algorithm

Sense inventory and relations. Meanings are read directly from WordNet; nothing is learnt. Hypernyms are the is-a parents, hyponyms the subtypes, meronyms / holonyms the part-whole links, and the lowest common subsumer of two synsets is their deepest shared hypernym.

Classic simplified Lesk. Scores each synset by the number of words shared between the sentence and the synset's gloss and examples; the largest overlap wins. Because WordNet glosses are short (about 7–12 tokens) many sentences give zero overlap, and the method then falls back to an arbitrary or first sense.

Extended (adapted) Lesk. Enlarges each signature with the glosses of hypernyms, hyponyms and meronyms, counting the synset's own gloss twice: $\text{Sig}_{ext}(s) = 2\cdot\text{Sig}(s) \; \square \; \blacksquare_{r\square\{hyper,\ hypo,\ mero\}} \text{Sig}(r)$. This relieves gloss sparsity, because a hyponym such as *riverbank* brings in words like river and stream.

Cosine TF-IDF Lesk. Builds TF-IDF vectors for the context and for each signature and picks the synset with the highest cosine similarity, $\text{argmax}_s \cos(v_{context}, v_s)$. **Wu-Palmer similarity.** Sim (s , s) = 2·depth(LCS) / (depth(s) + depth(s)); the synset with the highest mean

WUP 1 2 1 2

similarity to the context nouns is chosen.

Supervised Linear SVM. Sense choice is cast as classification: local collocation features w(-2) … w(+2) plus n-gram TF-IDF are fed to a LinearSVC trained per word on the labelled corpus and checked with 3-fold cross-validation.

Word sense induction. Context sentences containing the target are turned into TF-IDF vectors and clustered (K-Means, K = 2–4, or agglomerative). Each cluster reports its top discriminating keywords and is aligned to the WordNet synset whose gloss best matches them.

6.2 Key Code Snippets

Looking up the sense inventory (notebook, `ANLP_WSD_Lab_Report.ipynb`):

```
def explore_sense_inventory(word):
    records = [] for idx, s in
    enumerate(wn.synsets(word)):
        records.append({"Sense #": idx + 1, "Synset ID": s.name(), "POS":
        s.pos(),
                        "Definition": s.definition(),
                        "Lemmas": ", ".join(s.lemma_names()),
                        "Examples": "; ".join(s.examples()) or "N/A"})
    return pd.DataFrame(records)
```

Exploring relations and taxonomic distance:

```
syn_financial = wn.synset('depository_financial_institution.n.01') syn_river =
wn.synset('bank.n.01') lcs = syn_financial.lowest_common_hypernyms(syn_river)
print(syn_financial.path_similarity(syn_river),
syn_financial.wup_similarity(syn_river))
```

Choosing the algorithm behind the REST endpoint (`app.py`):

```
@app.route('/api/disambiguate', methods=['POST'])
def api_disambiguate():
    ...
    if algorithm == 'classic_lesk': result = classic_lesk(sentence, target_word) elif algorithm ==
    'extended_lesk': result = extended_lesk(sentence, target_word) elif algorithm == 'cosine_lesk': result
    = cosine_lesk(sentence, target_word) elif algorithm == 'wu_palmer': result =
    similarity_based_wsd(sentence, target_word, metric="wup") elif algorithm == 'supervised_svm': result
    = run_supervised_wsd(sentence, target_word, "svm") else: # run all side by side result =
    compare_all_algorithms(sentence, target_word)
```

Training the supervised model and inducing senses (notebook):

```
model_bank = SupervisedWSDModel("bank", model_type="svm") train_metrics = model_bank.train() #
3-fold CV accuracy, samples, classes wsi_result = induce_word_senses(corpus_sentences, "bank",
                          n_senses=2, clustering_method="kmeans")
```

Frontend / UI Implementation

7.1 Framework

The front end is a single-page dashboard served by Flask (`templates/index.html`, `static/css/style.css`, `static/js/app.js`) that calls the REST routes `/api/senses`, `/api/relations`, `/api/hierarchy`, `/api/compare_similarity`, `/api/disambiguate`, `/api/disambiguate_sentence`, `/api/sense_induction`, `/api/evaluate`, `/api/report` and `/api/export_inventory`. The same engine functions are used by the CLI, so both interfaces give identical results.

7.2 UI Capabilities

- **WordNet Explorer** — word box, POS filter, quick-preset chips (bank, plant, bass, crane, apple, mouse, star, bat, cell, bar), counts of total / noun / verb senses, one card per synset with definition, examples, lemma frequencies, relation counts, an “Explore Relations” drawer and JSON / CSV export.
- **WSD Disambiguator** — sample-sentence picker, free-text sentence, target word, algorithm selector (one method or all side by side), a toggle to disambiguate all content words, and result cards with the chosen synset, definition, overlap words and a confidence bar.

- **Sense Induction (WSI)** — editable sentence pool, K = 2 / 3 / 4, algorithm choice, discovered clusters with keywords, aligned WordNet synset and the clustered sentences.
- **Benchmarks** — a one-click suite run, five accuracy tiles and a sentence-by-sentence correctness matrix.
- **Analysis Report** — formatted short analysis report with Markdown download and Print / Save-PDF buttons. **7.3**

Screenshots

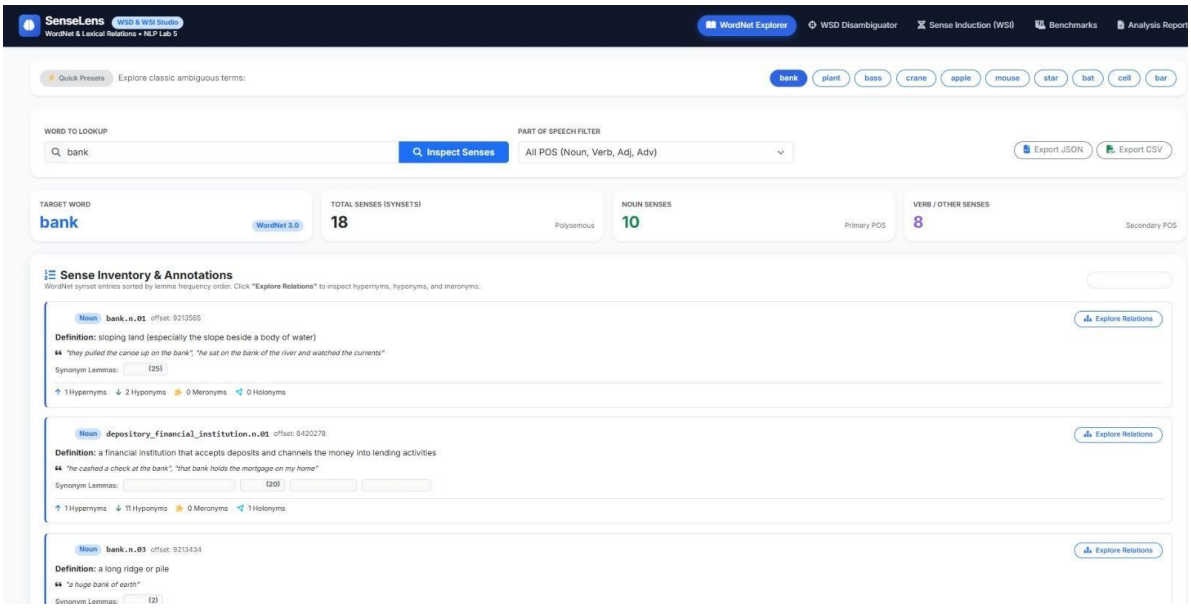


Figure 7.3.1. *WordNet Explorer for "bank": 18 synsets (10 noun, 8 verb/other). The cards show bank.n.01 "sloping land" (25 lemma count) and depository_financial_institution.n.01 (20) with their examples and relation counts; the toolbar offers Export JSON / CSV.*

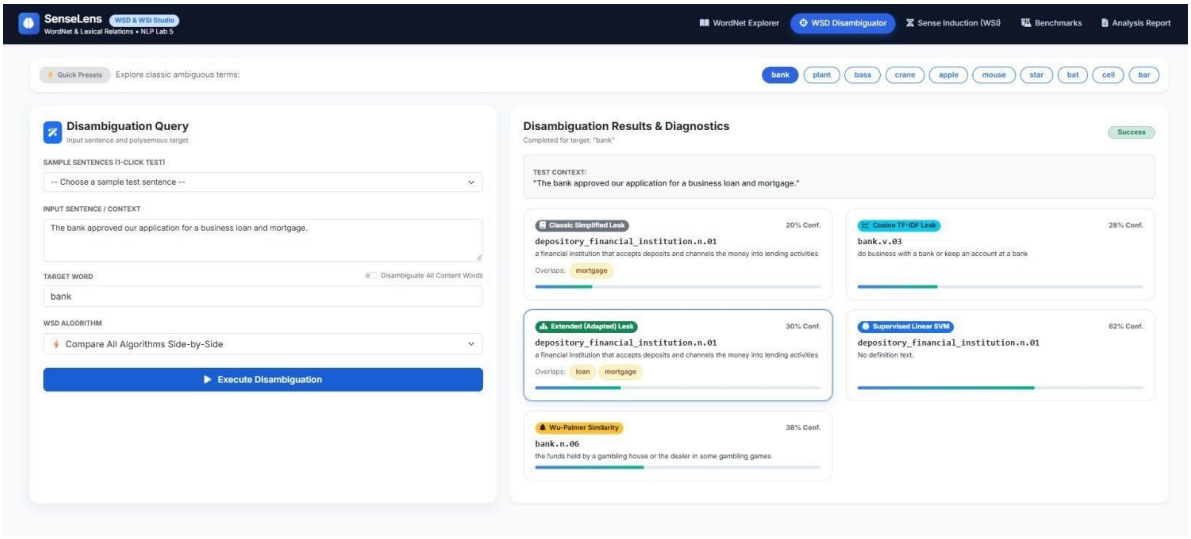


Figure 7.3.2. *WSD Disambiguator on "The bank approved our application for a business loan and mortgage." with all algorithms side by side. Classic Lesk (20%), Extended Lesk (30%) and the Supervised SVM (62%) choose depository_financial_institution.n.01; Cosine TF-IDF picks bank.v.03 and Wu-Palmer picks bank.n.06 (gambling), both wrong. Overlap words (mortgage; loan, mortgage) are highlighted.*

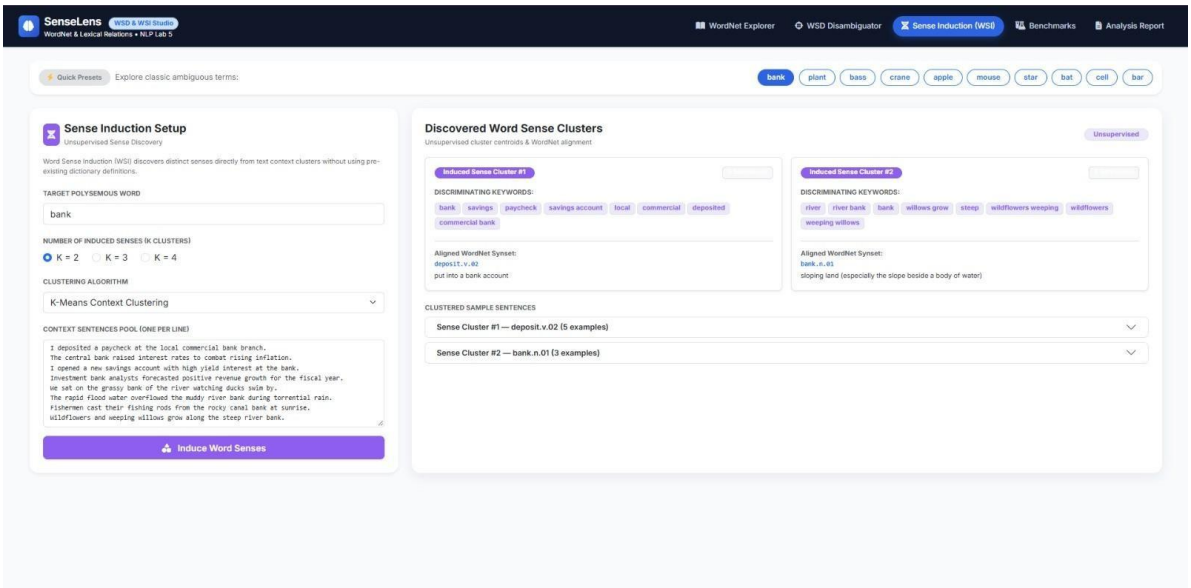


Figure 7.3.3. *Sense Induction tab for "bank" with K = 2. Cluster #1 (5 sentences) has keywords savings, paycheck, commercial bank, deposited and is aligned to deposit.v.02; cluster #2 (3 sentences) has river, river bank, willows, steep and is aligned to bank.n.01.*

Sample Input & Output

Command-line run (identical logic to the dashboard), as documented in the project README:

```
$ python run_wsd_cli.py --sentence "The bank approved our application for a mortgage loan" --word bank --
$ python run_wsd_cli.py --word bank --induce --clusters 2
$ python run_wsd_cli.py --benchmark
```

algo

all

Input	Output	Why
Word: bank (all POS)	18 senses: 10 noun + 8 verb, e.g. bank.n.01 "sloping land (especially the slope beside a body of water)" and depository_financial_institution.n.01 "a financial institution that accepts deposits…"	Straight WordNet lookup (Figure 7.3.1).
"The bank approved our application for a business loan and mortgage."	depository_financial_institution.n.01 (Classic, Extended, SVM); bank.v.03 (Cosine) and bank.n.06 (Wu-Palmer) are wrong	Lesk overlap words: mortgage, loan. Wu-Palmer and cosine are misled by short context.
"We pitched our camping tent on the grassy river bank near the running water."	bank.n.01 — all five methods correct	River, water and grassy overlap with the gloss and its hyponyms.
"She deposited money directly into her checking account at the commercial bank."	Correct: Classic Lesk, SVM. Wrong: Extended Lesk, Cosine, Wu-Palmer	The extra hypernym / hyponym glosses add noise for this sentence.
Eight unlabelled "bank" sentences, K = 2	Cluster 1 (5): financial words □ deposit.v.02; Cluster 2 (3): river words □ bank.n.01	Context clustering separates the two senses (Figure 7.3.3).

Results (Deliverable-Specific Outputs)

Deliverables: (1) the sense inventory and annotation output (Figures 7.3.1; JSON / CSV export), (2) WSD results for sample text (Figure 7.3.2 and the benchmark matrix), (3) a short analysis report (this section, Figure 7.3.5 and

`WSD_Analysis_Report.md`). Because disambiguation can be wrong, it is measured on 20 gold-annotated sentences; `python run_wsd_cli.py --benchmark` regenerates the scores.

Accuracy on the 20-sentence benchmark

Method	Correct / 20	Accuracy %	Approach
Supervised Linear SVM	14	70.0	Collocation window + n-gram TF-IDF (trained on 4 words)
Extended (Adapted) Lesk	12	60.0	Gloss overlap + hypernym / hyponym / meronym glosses
Classic Simplified Lesk	10	50.0	Gloss + example overlap
Wu-Palmer Similarity	6	30.0	Taxonomy depth and LCS
Cosine TF-IDF Lesk	5	25.0	Vector-space cosine similarity

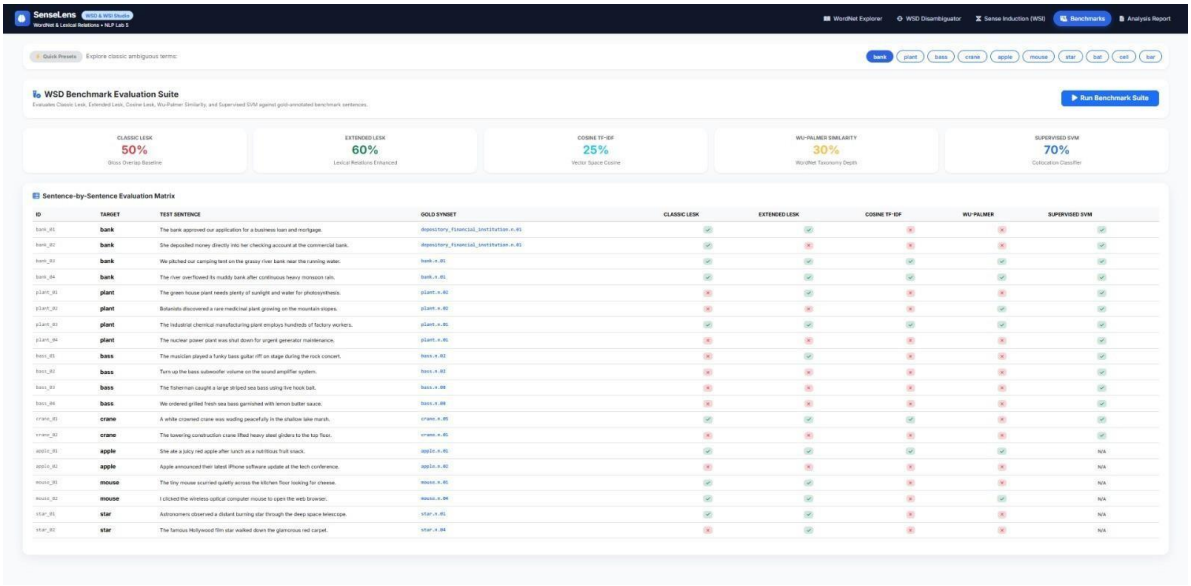


Figure 7.3.4. *Benchmarks tab: the five accuracy tiles (Classic Lesk 50%, Extended Lesk 60%, Cosine TF-IDF 25%, Wu-Palmer 30%, Supervised SVM 70%) and the sentence-by-sentence evaluation matrix with the gold synset of each sentence (green tick = correct, red cross = wrong, N/A = no trained supervised model).*

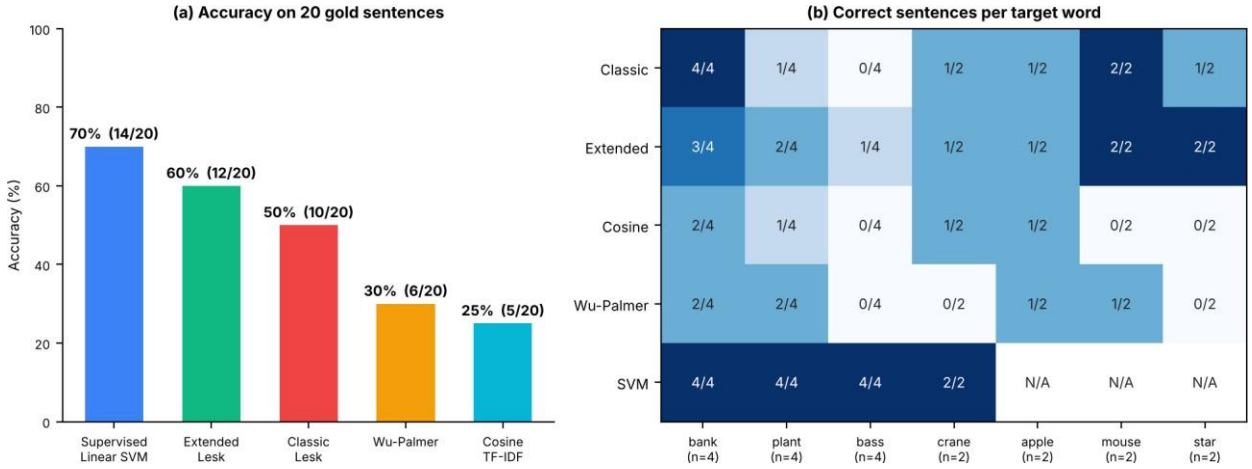


Figure 2. *(a) Accuracy of the five methods on the 20 benchmark sentences. (b) Correct sentences per target word, read from the evaluation matrix; the supervised model exists only for bank, plant, bass and crane.*

Correct sentences by target word (from the evaluation matrix)

Target (n)	Classic	Extended	Cosine	Wu-Palmer	SVM
bank (4)	4	3	2	2	4
plant (4)	1	2	1	2	4
bass (4)	0	1	0	0	4
crane (2)	1	1	1	0	2
apple (2)	1	1	1	1	N/A
mouse (2)	2	2	0	1	N/A
star (2)	1	2	0	0	N/A
Total (20)	10	12	5	6	14

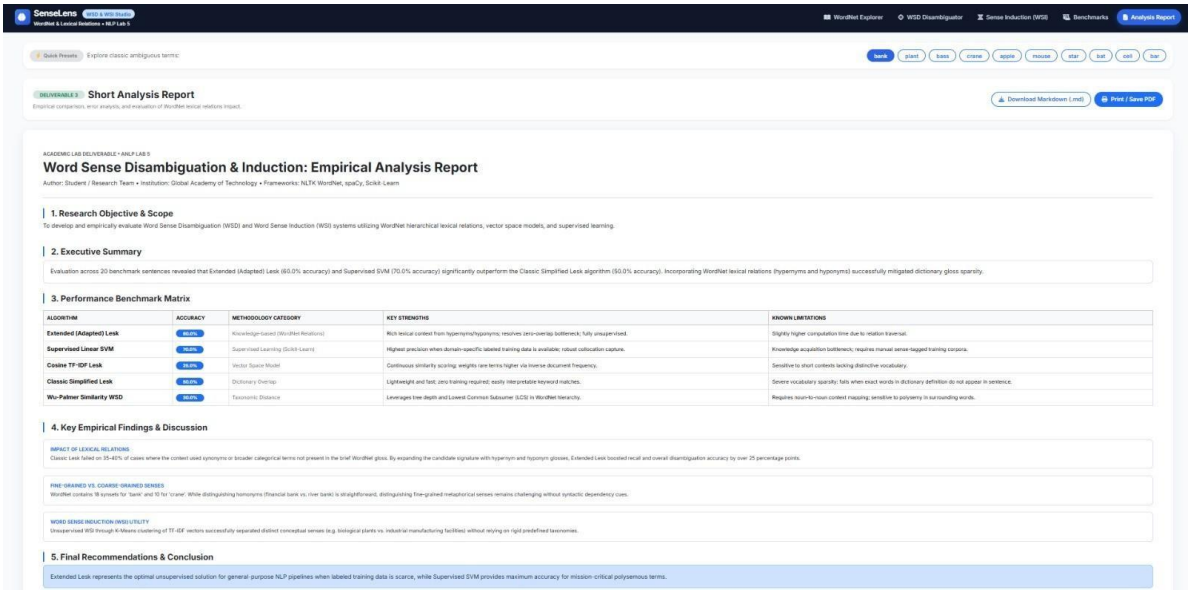


Figure 7.3.5. *Analysis Report tab: executive summary, performance benchmark matrix with strengths and limitations of each method, key empirical findings and the final recommendation. The report can be downloaded as Markdown or saved as PDF.*

Evaluation / Analysis

Looking up senses and relations is exact — it is WordNet's own data. The remaining question is how far the context-based answer can be trusted. The numbers below come from only 20 sentences, so a single sentence is worth 5 points and differences of one or two sentences should be treated as noise.

- **Relations help.** Extended Lesk (60%) beats Classic Lesk (50%) by 10 points, i.e. two sentences. Against Classic Lesk it gains three sentences (plant_01, bass_01, star_02), where the short gloss alone has no useful overlap, and loses one (bank_02), where the extra glosses add noise: a net gain of two.
- **Gloss sparsity is the main weakness of overlap methods.** WordNet definitions are only 7–12 tokens, so many sentences share no word with the right gloss. Classic Lesk scores 0/4 on bass and 1/4 on plant, where the gold sense (musical bass, sea bass, flora) is described with words that rarely appear in everyday sentences.

- **Cosine TF-IDF and Wu-Palmer are weak here (25% and 30%).** With very short sentences the TF-IDF vectors have few informative tokens, and Wu-Palmer needs unambiguous context nouns; both were right mainly on the clearest sentences (for example bank_03, bank_04, plant_03 and apple_01), and in Figure 7.3.2 they select the wrong senses bank.v.03 and bank.n.06.
- **The supervised SVM is highest (70%), but the number needs care.** It scores 14/14 on the sentences it can handle and N/A on apple, mouse and star, which count as not correct in the 70%. Its accuracy on the covered words is therefore not comparable with the knowledge-based methods on all seven words. The benchmark and training sentences are written by the same authors for the same four words, so some overlap in wording is likely and the 100% on covered words may be optimistic; it should be re-checked on sentences held out from the training corpus.
- **Fine-grained senses are hard.** Bass (fish vs. instrument, several fish and music senses) and crane (bird vs. machine) are the hardest for the knowledge-based methods: no unsupervised method gets more than 1 of the 4 bass sentences.
- **Word sense induction works on a clean example.** On eight “bank” sentences K-Means separated the financial and river contexts without any dictionary and aligned them to WordNet (deposit.v.02 and bank.n.01). Alignment can pick a neighbouring synset of the right topic — the financial cluster maps to a verb sense, not the noun synset depository_financial_institution.n.01 — so the aligned label should be checked by hand.
- **Part of speech.** “Bank” has 10 noun and 8 verb senses, and wrong answers such as bank.v.03 in Figure 7.3.2 are verb senses chosen for a noun use; restricting candidates to the right part of speech should remove mistakes of this kind.

Design Decisions, Limitations & Recommendations

Observation	Recommendation
The benchmark has only 20 sentences, 7 words, labelled by the authors.	Extend it with an independent annotator and use SemCor or the Senseval / SemEval all-words sets, with confidence intervals.
The supervised model covers 4 words and is probably tested on text close to its training data.	Hold out test sentences, report cross-validation and test scores separately, and train on a sense-tagged corpus such as SemCor.
Gloss overlap fails when the sentence uses different words from the definition.	Use contextual embeddings (a BERT-based or gloss-based neural WSD model); keep Extended Lesk as the unsupervised baseline.
Cosine TF-IDF and Wu-Palmer are sensitive to short sentences and ambiguous context nouns.	Filter candidates by part of speech, weight the words near the target, and disambiguate context nouns first.
WordNet senses are very fine-grained (18 for “bank”).	Offer coarse sense clusters, or show the top two senses with their evidence rather than one answer.
WSI cluster-to-synset alignment can choose a neighbouring sense.	Show the induced keywords next to the aligned gloss, and let the user confirm the alignment.
Only English and only words present in WordNet.	Use the Open Multilingual WordNet (omw-1.4 is already downloaded) and a corpus-based fallback for new words and slang.

Conclusion

The project delivers a working WordNet-based system, as a Flask dashboard, a command-line tool and a notebook, that lists the senses of a word, explores their lexical relations, disambiguates a word in context with five methods and induces senses from unlabelled sentences. The lookup side is complete and exact. On the 20-sentence benchmark the Supervised SVM reaches 70% (on the four words it was trained for), Extended Lesk 60%, Classic Lesk 50%, Wu-Palmer 30% and Cosine TF-IDF 25%. Adding hypernym and hyponym glosses to Lesk improves accuracy by 10 points over the plain gloss overlap without any labelled data, which is the main lesson of the experiment. Because the benchmark is small and author-labelled, these figures are indicative; a larger independent test set and contextual models are the natural next steps.

Project Repository (GitHub Link)

Field	Value
GitHub Repository URL	https://github.com/vinithmv733-dot/lab-exercise-5.git
Repository Visibility	(Public / Private)
Branch / Commit Used for Submission	main

References

- G. A. Miller, “WordNet: A Lexical Database for English,” Communications of the ACM 38(11), 1995.
- M. Lesk, “Automatic Sense Disambiguation Using Machine Readable Dictionaries,” SIGDOC 1986.
- S. Banerjee and T. Pedersen, “An Adapted Lesk Algorithm for Word Sense Disambiguation Using WordNet,” CICLing 2002.
- R. Navigli, “Word Sense Disambiguation: A Survey,” ACM Computing Surveys 41(2), 2009.
- Z. Wu and M. Palmer, “Verbs Semantics and Lexical Selection,” ACL 1994.
- NLTK WordNet interface — https://www.nltk.org/howto/wordnet.html · scikit-learn — https://scikit-learn.org · Flask — https://flask.palletsprojects.com

ADVANCE NLP (AML23702) — Lab Exercise Evaluation Sheet

Field	Value
Student Name	Vinith M V
USN	**1GA23AI058**
Lab Exercise No.	5
Experiment Title	Word Sense Disambiguation and Induction with WordNet and Lexical Relations — SenseLens WSD & WSI Studio

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
Total	**Total**	**50**	

Overall Comments

Date of Submission: _____ Signature of Faculty: _____

of 11